

Scheduling di processi e thread

Antonio Pecchia

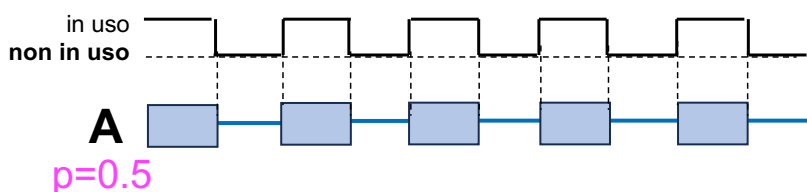
antonio.pecchia@unisannio.it

Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



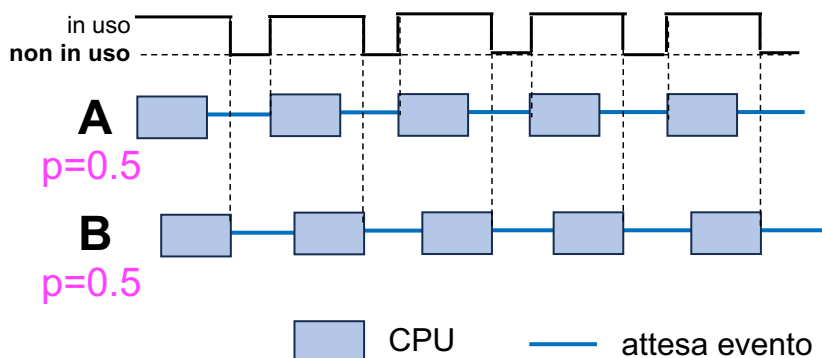
UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

Modello multiprogrammazione (approssimato)



Utilizzo CPU

$$(1 - 0.5) * 100 = 50.0\%$$



$$0.5^2 = 0.25$$

$$0.5^3 = 0.125$$

$$0.5^4 = 0.0625$$

$$(1 - 0.5^2) * 100 = 75.0\%$$

$$(1 - 0.5^3) * 100 = 87.5\%$$

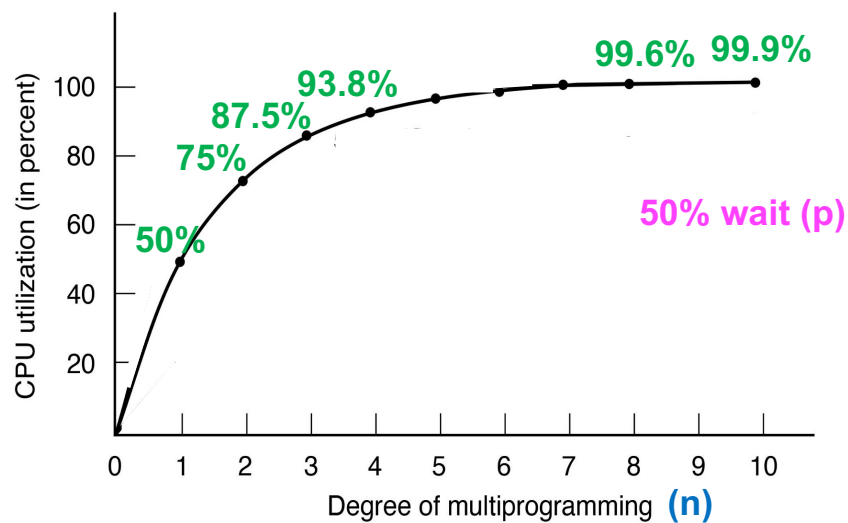
$$(1 - 0.5^4) * 100 = 93.8\%$$

...

probabilità che *tutti* gli n processi
stiano in attesa evento

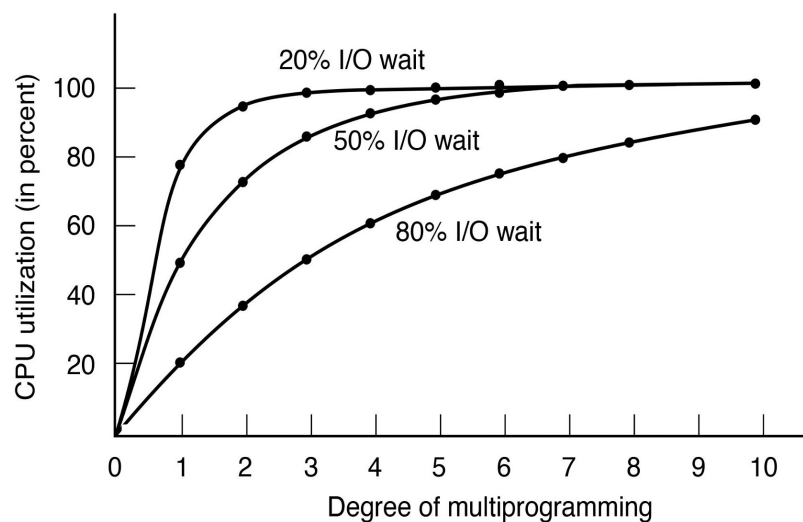
Formula
utilizzo CPU $(1 - p^n) * 100$

Modello multiprogrammazione* (approssimato)



*Cap. 2 par. 2.1.7

Modello multiprogrammazione* (approssimato)



*Cap. 2 par. 2.1.7

SCHEDULING DI PROCESSI E THREAD

Cap. 2: par. 2.5.1

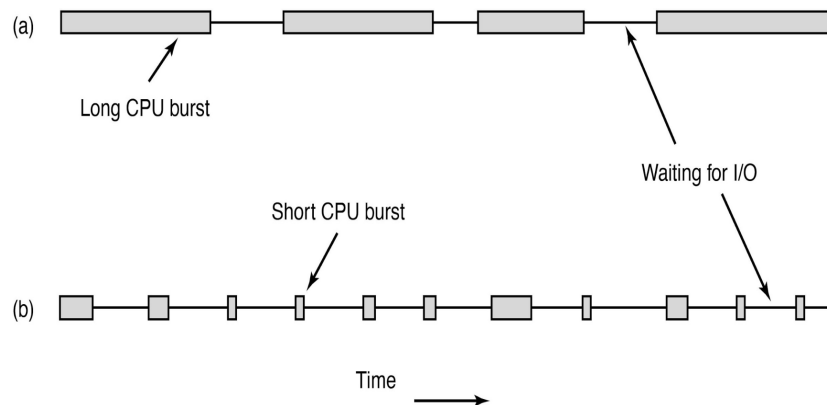
Scheduler

- **Scheduler**: parte del SO preposta all'assegnazione di risorse a favore dei processi:
 - il processo è scelto con un **algoritmo di scheduling**.
- "Focus" della lezione: **scheduler della CPU** (o scheduler a breve termine):
 - selezione tra 2 o più processi in stato **pronto**;
- Quando il **kernel gestisce i thread**, lo scheduling della CPU è fatto per thread.

Sidenote

- Tutti i processi alternano *burst* (fasi intense) di CPU e I/O

Processo
CPU bound



Processo
I/O bound

Si effettua lo scheduling quando...

- un processo termina;
- un processo si blocca in attesa di un evento;
- viene creato un processo;
- si verifica un interrupt.

² un processo *running* può essere interrotto e spostato in *ready* dal SO (per es., allo scadere di un tempo massimo prefissato);

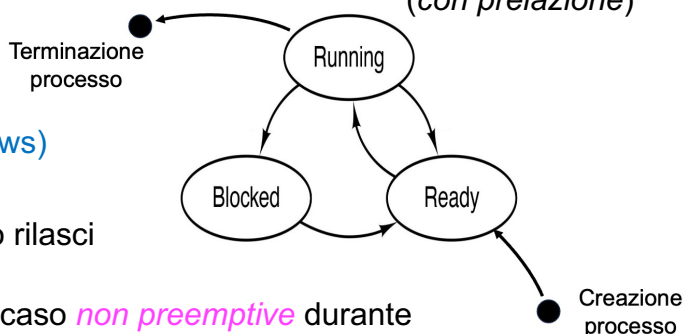
i SO moderni (inclusi macOS, Linux, Windows) usano algoritmi di scheduling **preemptive**.

¹ al massimo si può "sperare" che il processo rilasci *volontariamente* la CPU;

³ richiede un timer interrupt (tipicamente, nel caso *non preemptive* durante un timer interrupt non sono prese decisioni di scheduling.).

scheduling non preemptive¹
(senza prelazione)

scheduling preemptive^{2,3}
(con prelazione)



Obiettivi di un algoritmo di scheduling

Equità – dare a ogni processo un'equa condivisione della CPU

Imposizione della policy – assicurarsi che la policy dichiarata sia attuata

Bilanciamento – tenere impegnate tutte le parti del sistema

Throughput – massimizzare il numero di job* per ora (o anche *numero di job* terminati per unità di tempo*)

Tempo di turnaround – ridurre al minimo il tempo tra il momento in cui un job* viene sottoposto e quello in cui termina

Utilizzo della CPU – mantenere la CPU sempre impegnata

Tempo di risposta – rispondere alle richieste rapidamente

Adeguatezza – far fronte alle aspettative dell'utente

Rispetto delle scadenze – evitare la perdita di dati

Prevedibilità – evitare il degrado della qualità nei sistemi multimediali

Tempo di attesa – tempo che il processo ha speso nello stato ready

* job o processi

tempo che trascorre dall'istante della sottomissione di una richiesta fino all'istante in cui la risposta inizia ad essere ricevuta

Esiste un algoritmo di scheduling perfetto?

- ❑ **NO**. Gli obiettivi sono tanti, *interdipendenti*, e, talvolta, anche in *conflitto* tra di essi:
 - un algoritmo di scheduling implica un compromesso che tenga conto anche della tipologia di sistema / dominio applicativo.

Obiettivi generali	Equità	Throughput	Sistemi batch
	Imposizione della policy	Tempo di turnaround	
	Bilanciamento	Utilizzo della CPU	Sistemi interattivi
		Tempo di risposta	
		Tempo di attesa	Sistemi real-time
		Adeguatezza	
		Rispetto delle scadenze	
		Prevedibilità	

ALGORITMI DI SCHEDULING

Cap. 2: par. 2.5.2 e 2.5.3 (fino a Shortest Process Next incluso)

First-come, First-served (FCFS)

- I processi sono assegnati alla CPU nell'ordine di arrivo:

	<u>Process</u>	<u>Service Time</u>
• ordine di arrivo P_1, P_2, P_3 :	P_1	24
	P_2	3
	P_3	3



- tempo di attesa: $P_1 = 0; P_2 = 24; P_3 = 27$
- tempo di attesa medio: $(0 + 24 + 27)/3 = 17$

First-come, First-served (FCFS)

- Un processo "corto" può aspettare molto:
 - **convoy effect**: i processi I/O-bound devono attendere il completamento di quelli CPU-bound.

	<u>Process</u>	<u>Service Time</u>
• ordine di arrivo P_2, P_3, P_1 :	P_2	3
	P_3	3
	P_1	24

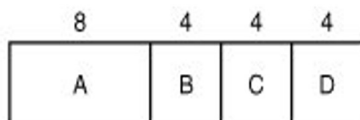


- tempo di attesa: $P_2 = 0; P_3 = 3; P_1 = 6;$
- tempo di attesa medio: $(0 + 3 + 6)/3 = 3$

Shortest job first

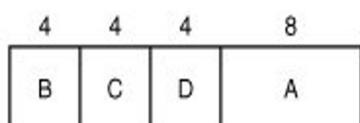
- Lo scheduler preleva, *per primo*, il job più breve:
 - **assunzione**: i tempi di esecuzione sono *noti in anticipo*.

	<u>Process</u>	<u>Service Time</u>
• esecuzione secondo ordine di arrivo:	A	8
	B	4
	C	4
	D	4



attesa media
 $(0+8+12+16)/4=9$

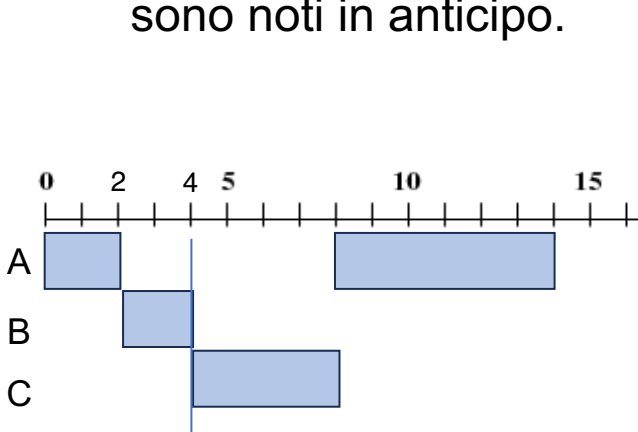
- esecuzione **shortest job first**:



attesa media
 $(0+4+8+12)/4=6$

Shortest remaining time next

- Versione **con prelazione** di shortest job first.
 - **assunzione**: (anche in questo caso) i tempi di esecuzione sono noti in anticipo.



<u>tempo di arrivo</u>	<u>Process</u>	<u>Service Time</u>
0	A	8 6*
2	B	2
4	C	4

* *remaining time* calcolato al tempo 2 (quando A aveva già eseguito per 2 unità di tempo).

Shortest process next (SPN)

- Come applicare i concetti visti quando **non sono noti*** i tempi di servizio?
- Necessità di "prevedere" quale, dei **processi ready**, occuperà il processore per il più breve tempo:
 - la stima è basata sui **precedenti** burst di CPU;
 - viene selezionato il processo con il più breve (**shortest**) burst di CPU **stimato**;
 - può avere sia una implementazione *non preemptive*, sia *preemptive*.

*scenario tipico dei sistemi interattivi

SPN: stima del prossimo CPU burst

- La previsione è realizzata tramite un meccanismo di **exponential averaging** (media esponenziale):

predizione della durata del $(n+1)^{th}$ CPU burst

durata effettiva del n^{th} CPU burst

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

$0 \leq \alpha \leq 1$

predizione della durata del n^{th} CPU burst

Scelta comune:

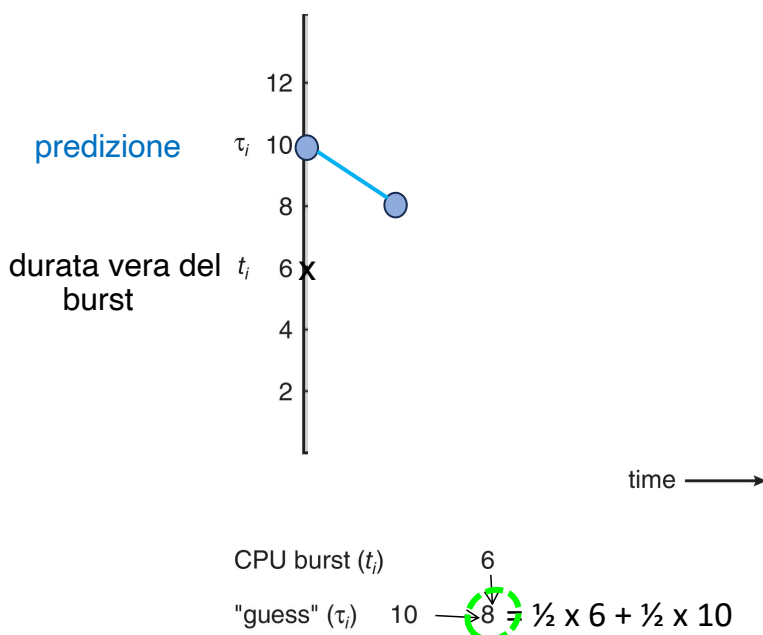
$$\alpha = \frac{1}{2}$$

Casi limite:

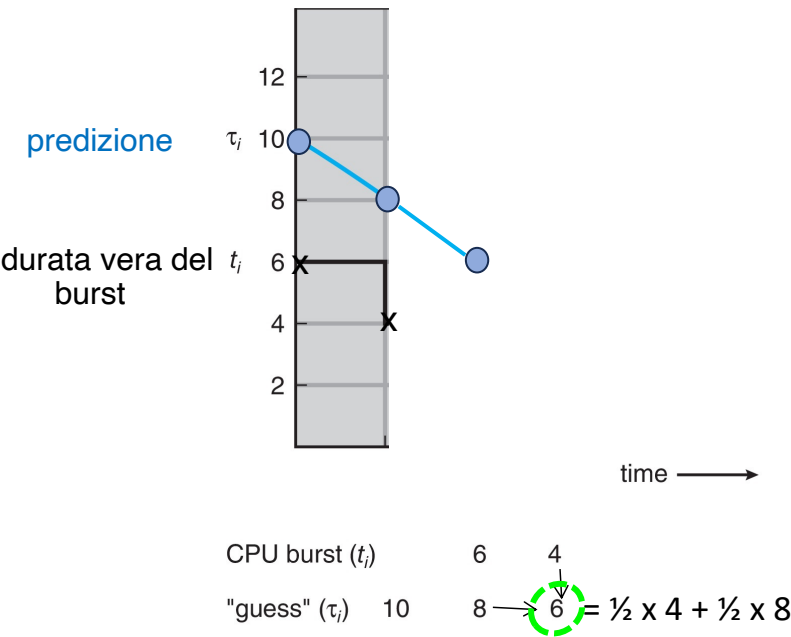
$\alpha = 0$ (il CPU burst t_n non pesa nel calcolo);

$\alpha = 1$ (conta solo il CPU burst t_n).

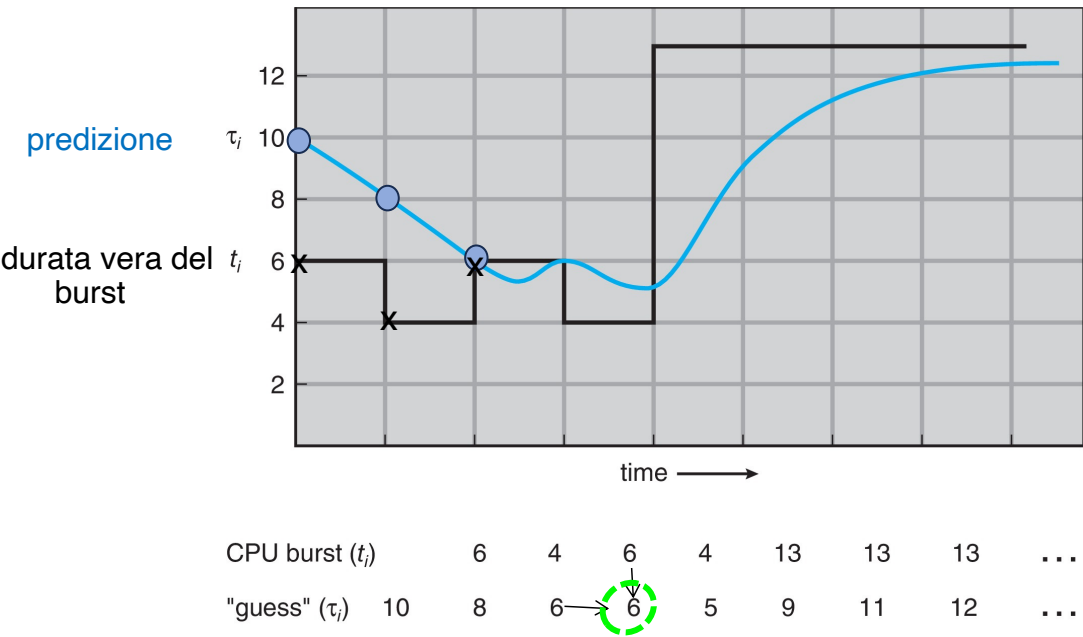
Predizione del next burst



Predizione del next burst



Predizione del next burst

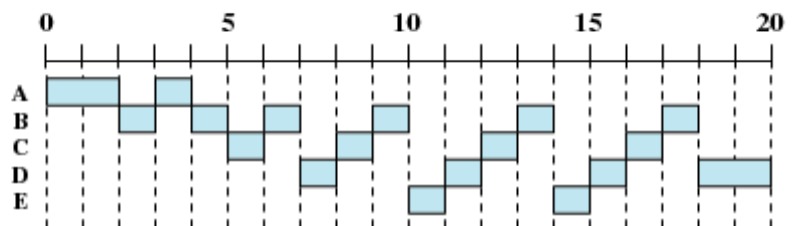


Round robin

- Ad ogni processo è assegnato un **quanto** di tempo:
 - se alla fine del quanto il processo è ancora in esecuzione, la CPU viene **prelazionata** in favore di un altro processo.

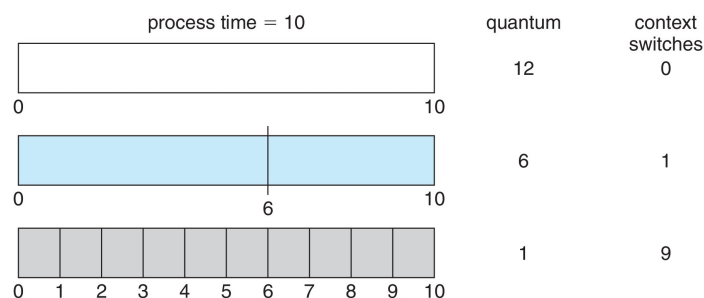
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Round-Robin
(RR), $q = 1$



Round robin: durata del quanto

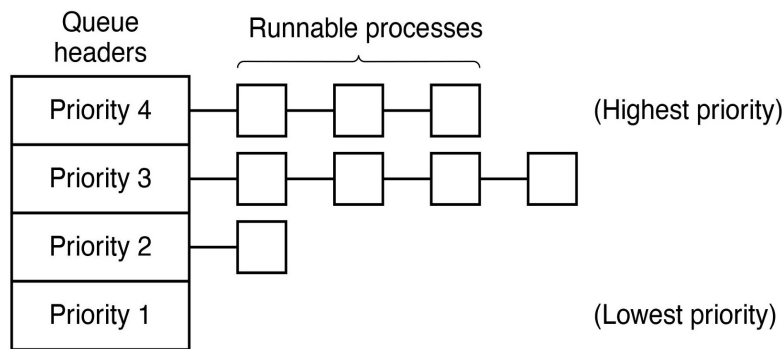
- La durata del quanto (q) è **critica**:
 - q troppo **breve** $\rightarrow$ troppi context switch.
 - q troppo **lungo** $\rightarrow$ "degenera" in un FCFS;



- valore tipico:** quanto compreso tra i 20 e 50 *ms*

Scheduling a priorità

- **Schema tipico:** i processi sono raggruppati in **classi di priorità***; all'interno di ciascuna classe, si usa l'approccio round robin.



* la priorità è rappresentata, tipicamente, da un numero (intero) associato al processo

Scheduling a priorità

- **Versione "base":** esecuzione consentita al processo ready con la priorità più alta:
 - **problema:** **starvation** dei processi con minore priorità;
- Gestire le priorità non è semplice. **Soluzioni pratiche:**
 - riduzione della priorità del processo in esecuzione ad ogni clock interrupt;
 - riduzione "volontaria" della priorità dei propri processi da parte dell'utente (per es., comando UNIX **nice**);
 - continua slide successiva...

Scheduling a priorità

▪ ...

- assegnare priorità **maggiore** ai processi che *non* riescono ad utilizzare tutto il proprio quanto di tempo:
 - buona soluzione per i **processi I/O-bound**.
 - esempio: **priorità** = $1 / f$, dove f è la frazione dell'ultimo quanto usata dal processo.

$$q = 50$$

$$\text{usato dal processo} = 1$$

$$\text{priorità} = 1 / (1/50) = 50;$$

$$q = 50$$

$$\text{usato dal processo} = 25$$

$$\text{priorità} = 1 / (25/50) = 2;$$

Multilevel feedback queue

- È uno scheduler definito dai seguenti **parametri**:
 - numero di code;
 - algoritmo di scheduling per coda;
 - metodi usati per determinare l'*upgrade* di un processo;
 - metodi usati per determinare il *downgrade* di un processo;
 - metodi usati per stabilire la coda iniziale di un processo.
- Un processo può "spostarsi" tra le code.

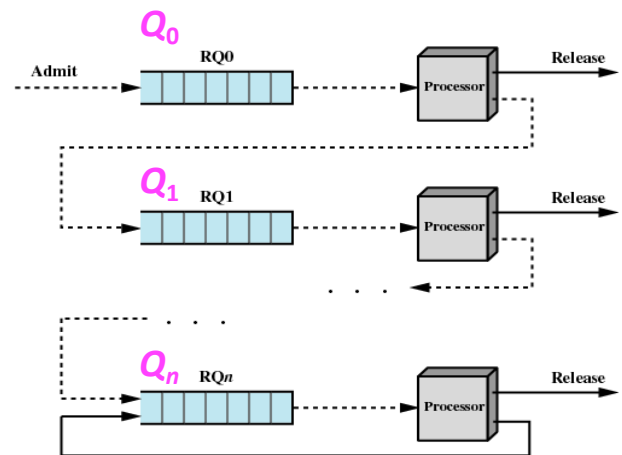
Multilevel feedback queue: un esempio

Tre code:

- Q_0 – RR; quantum = 8 milliseconds
- Q_1 – RR; quantum = 16 milliseconds
- Q_2 – FCFS

Scheduling:

- Un nuovo processo entra in Q_0 :
 - ▶ il processo riceve 8 ms; se non termina negli 8 ms, esso viene spostato in Q_1
- In Q_1 , il processo è servito ancora in RR e riceve ulteriori 16 ms:
 - ▶ se non termina, viene spostato in Q_2 .



mantiene i processi con prevalenza di I/O e i processi interattivi nelle code a priorità più elevata

Quadro riassuntivo

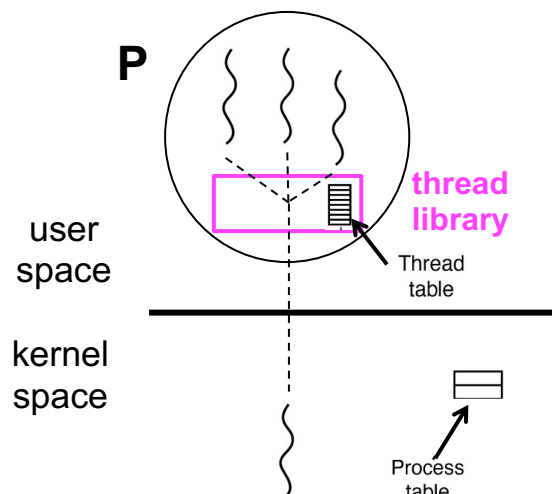
	Funzione di selezione	Modalità di decisione	Throughput	Tempo di risposta	Impatto sui processi	Attesa indefinita
FCFS	Ordine di arrivo	Non interrompente	Non enfatizzato	Potrebbe essere alto	Penalizza i processi brevi ed I/O bound	No
SRT	Min tempo rimanente	Interrompente (istante di arrivo)	Elevato	Buono	Penalizza i processi lunghi	possibile
SPN	Min burst stimato	In genere, non interrompente	Elevato	Tempi di risposta buoni per processi brevi	Penalizza i processi lunghi	possibile
Round Robin	Costante	Interrompente (quanto di tempo)	Può essere basso se il quanto è troppo breve	Tempi di risposta buoni per processi brevi	Politica fair	No
Feedback	Con retroazione	Interrompente (quanto di tempo)	Non enfatizzato	Non enfatizzato	Potrebbe favorire i processi I/O bound	possibile

SCHEDULING DEI THREAD

Cap. 2: par. 2.5.6

User-level thread

- Lo scheduler **del SO** sceglie il **processo** e gli passa il controllo per il suo quanto:
 - lo scheduler **interno al processo** (implementato dalla thread library) decide quale thread eseguire;
 - l'algoritmo di scheduling della library può essere uno qualsiasi di quelli descritti.



Kernel-level thread

- Lo scheduler **del SO** sceglie quale **thread** eseguire:
 - al thread viene attribuito il proprio **quanto** di tempo;
 - lo scheduler *può o non** tenere conto del processo a cui appartiene il thread.

***NOTA:** passare da un thread del **processo A** a un thread del **processo B** è *più costoso* che eseguire un altro thread del processo A.

